1.1.5. Implement … 12:30

You are given a set of integers and a target sum. Write a program that finds all subsets of the given set whose sum is equal to the target sum.



**Input Format:**

- The first line contains an integer, $n$ the number of elements in the set.
- The second line contains $n$ space-separated integers representing the elements of the set.
- The third line contains an integer, $sum$ the target sum.

**Output Format:**

- Print all subsets whose sum is equal to the given target sum. Each subset should be printed in a new line, and the elements within the subset should be separated by a space.
- If no such subset exists, print "-1"
- The subsets should be printed in reverse order of their discovery (i.e., the last found subset should be printed first).

**Example 1:**

**Input:**

```
3
1 2 1
3
```

**Output:**

```
2 1
1 2
```

**Example 2:**

**Input:**

Explorer

CTC64796.c

Submit

Debugger

```c
#include <stdio.h>
#include <stdlib.h>

int n;
int elements[20];
int target_sum;

int **subsets;
int *subset_sizes;
int subset_count = 0;

int current_subset[20];
int current_size = 0;

void backtrack(int index, int current_sum) {
    if (current_sum == target_sum) {

subsets[subset_count] = (int *)malloc(current_size * sizeof(int));
        for (int i = 0; i < current_size; i++) {

subsets[subset_count][i] = current_subset[i];
        }

subset_sizes[subset_count] = current_size;
        subset_count++;
        return;
    }

    if (index == n || current_sum > target_sum) {
        return;
```

‹ Prev Reset Submit Next ›

```
6
3 34 4 12 5 2
30
```

**Output:**

```
-1
```

**Constraints:**

- $1 \leq n \leq 20$
- $1 \leq element \leq 100$
- $1 \leq sum \leq 1000$

Sample Test Cases +

```
current_subset[current_size++] = elements[index];
    backtrack(index + 1, current_sum + elements[index]);
    current_size--;

    backtrack(index + 1, current_sum);
}

int main() {
    if (scanf("%d", &n) != 1) return 0;

    for (int i = 0; i < n; i++) {
        scanf("%d", &elements[i]);
    }
    scanf("%d", &target_sum);

    subsets = (int **)malloc((1 << n) * sizeof(int *));
    subset_sizes = (int *)malloc((1 << n) * sizeof(int));

    backtrack(0, 0);

    if (subset_count == 0) {
        printf("-1\n");
    } else {
        for (int i = subset_count - 1; i >= 0; i--) {
            for (int j
```

```
        subset_sizes[i]; j++) {
55
        printf("%d ", subsets[i]
        [j]);
56              }
57
        printf("\n");
58
        free(subsets[i]);
59          }
60      }
61
62      free(subsets);
63      free(subset_sizes);
64      return 0;
65  }
66
67
```